SQL SERVER PERFORMANCE SERIES

Indexes, Execution Plans & Operators

An end-to-end, hands-on reference with live examples and real-time tuning scenarios on AdventureWorks2019

PMSQLDBA · SQL DBA Champs

Platform: Microsoft SQL Server 2019 / 2022 · Sample DB: AdventureWorks2019
Production-grade DBA reference library

Table of Contents

How to Use This Guide 4
What you need 4
How to read it 4
Part 1 — Indexes from the Ground Up 5
1.1 How data is physically stored: pages and extents 5
1.2 The B-Tree: the shape of a rowstore index 5
1.3 Heaps — a table with no clustered index 6
1.4 Clustered index — the table itself, sorted 8
1.5 Nonclustered index — a sorted copy of selected columns 8
1.6 Composite indexes and key-column order 10
1.7 Covering indexes and INCLUDE columns 10
1.8 Unique indexes 12
1.9 Filtered indexes 12
1.10 Columnstore indexes — for analytics 12
1.11 Other index types you should recognize 14
1.12 Fill factor, fragmentation, and maintenance 14
1.13 Finding missing, unused, and duplicate indexes 15
Part 2 — Execution Plans 17
2.1 The optimizer in one paragraph 17
2.2 Estimated vs. Actual plans 17
2.3 How to read a graphical plan 17
2.4 Operator properties that matter 18
2.5 Cardinality estimation, statistics, and parameter sniffing 18
2.6 Plan cache and reuse 19
Part 3 — Plan Operators, Explained 20
3.1 Data-access operators: scans vs. seeks 20
3.2 Lookups: Key Lookup and RID Lookup 20
3.3 Physical join operators 22
Nested Loops 22
Merge Join 22
Hash Match 22
3.4 Sort, aggregate, and spool operators 24

 © PMSQLDBA

3.5 Parallelism and remaining operators 24
Part 4 — Real-Time Tuning Scenarios 26
Scenario 1 — A dashboard query doing a Key Lookup on a fat arrow 26
Scenario 2 — Implicit conversion silently killing a seek 26
Scenario 3 — Non-SARGable date filter forcing a scan 28
Scenario 4 — Parameter sniffing: fast for one customer, slow for another 28
Scenario 5 — A Sort spilling to tempdb 30
Scenario 6 — OLAP aggregation: rowstore scan vs. columnstore 30
Scenario 7 — A repeatable tuning workflow 32
Appendix — Quick Reference 33
A. Index decision cheat-sheet 33
B. Operator → likely fix 33
C. Essential tuning commands 33

 © PMSQLDBA

How to Use This Guide

This guide teaches three tightly-linked topics that every SQL Server DBA and developer must master together: **indexes** (the structures that make data fast to find), **execution plans** (the blueprint the engine chose), and **operators** (the individual steps inside that blueprint). You cannot fully understand one without the other two, so the guide builds them in order and then ties them together with real tuning scenarios.

What you need

- **SQL Server 2019 or later** (Developer or Enterprise edition recommended for ONLINE rebuilds and columnstore).
- The **AdventureWorks2019** sample database restored and online.
- **SQL Server Management Studio (SSMS)** or Azure Data Studio, so you can view graphical execution plans.

How to read it

1. **Part 1 — Indexes:** storage internals, then every index type with a live example.
2. **Part 2 — Execution Plans:** how the optimizer thinks and how to read the plan it produced.
3. **Part 3 — Operators:** what each operator does and what its presence signals.
4. **Part 4 — Real-time scenarios:** symptom → diagnose → fix → verify, end to end.
5. **Appendix:** cheat-sheets and the essential command set.

BEST PRACTICE — Run everything

Every code block is runnable against AdventureWorks2019.
Turn on
`SET STATISTICS IO ON;`
`SET STATISTICS TIME ON;`
and capture the **actual** execution plan (`Ctrl`+`M` in SSMS) before and after each change — measuring the difference is the whole point. Practising on your VMSQL01/02/03 lab first keeps experiments off production.

WARNING — Safe housekeeping

Several examples create helper objects (`dbo.SalesOrderDetail_Heap`, `dbo.SalesOrderDetail_CS`, and various indexes). Drop them when finished so you keep a clean AdventureWorks2019 copy: `DROP TABLE IF EXISTS dbo.SalesOrderDetail_Heap; DROP INDEX IF EXISTS ...`.

 © PMSQLDBA

Part 1 — Indexes from the Ground Up

An index is a physical data structure that lets SQL Server find rows without reading the whole table. The right index turns a query that scans millions of rows into one that touches a handful of pages. Before any specific index type makes sense, you have to understand how SQL Server physically stores data — because every index type is just a different arrangement of the same building blocks.

1.1 How data is physically stored: pages and extents

SQL Server never reads a single row from disk. It reads **pages**. Everything below is built on this fact.

Structure	Size	What it holds
Page	8 KB (8,192 bytes)	The smallest I/O unit. ~8,060 bytes usable for row data after headers.
Extent	8 contiguous pages = 64 KB	The unit of space allocation. Uniform (one object) or mixed (up to 8 objects).
Row	Varies	Stored inside a data page. A row cannot span pages (except LOB/overflow data).
Heap	Collection of pages	A table with NO clustered index — rows in no guaranteed order.
B-Tree	Root → intermediate → leaf	The structure of every clustered and nonclustered index (rowstore).

> **NOTE — Why "8 KB" matters**
> Every performance metric you will read in an execution plan — logical reads, physical reads, read-ahead — is counted in **pages**, not rows. When you see `logical reads 15423`, SQL Server touched 15,423 8-KB pages in the buffer pool. Reducing page reads is the core goal of most index tuning.

Turn on the two settings below in every tuning session. They report the page reads and CPU/elapsed time for each statement, which is how you objectively measure whether an index helped.

```
SET STATISTICS IO ON;    -- reports logical/physical page reads per table
SET STATISTICS TIME ON;  -- reports CPU and elapsed time per statement

-- Run any query here, then read the Messages tab in SSMS.
-- Example output:
--   Table 'SalesOrderDetail'. Scan count 1, logical reads 1246, physical reads 0
--   CPU time = 15 ms,  elapsed time = 210 ms
```

1.2 The B-Tree: the shape of a rowstore index

Every clustered and nonclustered rowstore index is a **balanced tree (B-Tree)**. It has three logical levels:

- **Root page** — a single page at the top. The entry point for every seek.
- **Intermediate levels** — one or more levels of navigation pages that point downward.

 © PMSQLDBA

- **Leaf level** — the bottom, where the actual data lives (for a clustered index) or where the key + row pointer lives (for a nonclustered index).

Because the tree is balanced, the number of page reads to find any single row is the same regardless of which row you want — it equals the depth of the tree. A table with tens of millions of rows typically has a B-Tree only 3–4 levels deep, which is why a **seek** is so cheap: 3–4 page reads to locate a row versus thousands of reads to scan.

> **TIP — Inspect the real tree depth**
> You can see the physical depth and page counts of any index with the DMV below. Run it against AdventureWorks2019 to prove to yourself how shallow these trees are.

```
USE AdventureWorks2019;
GO
SELECT  i.name              AS index_name,
        ips.index_type_desc,
        ips.index_depth,          -- number of B-Tree levels
        ips.index_level,          -- 0 = leaf
        ips.page_count,
        ips.record_count,
        CAST(ips.avg_fragmentation_in_percent AS DECIMAL(5,2)) AS frag_pct
FROM sys.dm_db_index_physical_stats
        (DB_ID(), OBJECT_ID('Sales.SalesOrderDetail'), NULL, NULL, 'DETAILED') ips
JOIN sys.indexes i
     ON i.object_id = ips.object_id AND i.index_id = ips.index_id
ORDER BY ips.index_id, ips.index_level DESC;
```

1.3 Heaps — a table with no clustered index

A **heap** is a table stored with no enforced order. Rows are placed wherever there is free space. SQL Server locates a heap row by its physical address, the **RID (Row Identifier)** = FileID : PageID : SlotNumber.

Heaps have no B-Tree at the base — just a chain of data pages tracked by allocation maps. To read specific rows from a heap you either scan every page or jump to a RID handed to you by a nonclustered index (a **RID Lookup**, covered in Part 3).

Create a heap and inspect it:

```
-- Build a heap copy of a table (SELECT INTO creates a heap by default)
SELECT * INTO dbo.SalesOrderDetail_Heap
FROM   Sales.SalesOrderDetail;

-- No clustered index exists -> index_id = 0 means HEAP
SELECT i.index_id, i.type_desc
FROM   sys.indexes i
WHERE  i.object_id = OBJECT_ID('dbo.SalesOrderDetail_Heap');
-- index_id 0  =  HEAP

-- Any lookup on a heap with no useful nonclustered index = full table scan
```

```
SET STATISTICS IO ON;
SELECT * FROM dbo.SalesOrderDetail_Heap WHERE SalesOrderID = 43659;
-- Table Scan, high logical reads
```

	Heap	Clustered table
Physical order	None — insertion order-ish	Ordered by the clustered key
Row locator	RID (physical address)	Clustered key value (logical)
Point lookup w/o index	Table Scan	Clustered Index Scan
Forwarding pointers	Yes — updates can create them, causing extra I/O	No such problem
Typical use	Staging / bulk-load landing tables	Almost every OLTP table

WARNING — Forwarding pointers

When a heap row grows too large to fit its original slot after an UPDATE, SQL Server moves it and leaves a **forwarding pointer** behind. Now every access chases an extra page. Heaps that get updated silently bloat their I/O over time. This is a major reason most transactional tables get a clustered index.

© PMSQLDBA

1.4 Clustered index — the table itself, sorted

A **clustered index defines the physical order of the table**. The leaf level of the clustered index *is* the data rows. A table can have only one clustered index because the data can be sorted only one way. In AdventureWorks2019, almost every table already has one — for example `Sales.SalesOrderDetail` is clustered on `(SalesOrderID, SalesOrderDetailID)`.

Look at an existing clustered index and how a seek uses it:

```
-- The clustered key on SalesOrderDetail
EXEC sp_helpindex 'Sales.SalesOrderDetail';
--  PK_SalesOrderDetail...  clustered, unique  (SalesOrderID, SalesOrderDetailID)

SET STATISTICS IO ON;

-- Seek: the predicate matches the LEADING clustered key column
SELECT SalesOrderID, SalesOrderDetailID, ProductID, LineTotal
FROM   Sales.SalesOrderDetail
WHERE  SalesOrderID = 43659;
-- Clustered Index Seek — a few logical reads only
```

The seek works because `SalesOrderID` is the **leading column** of the clustered key. SQL Server walks the B-Tree from the root down to the exact leaf page. Now compare with a predicate on the *second* key column only:

```
-- Predicate on the SECOND key column only -> cannot seek the tree
SELECT * FROM Sales.SalesOrderDetail
WHERE  SalesOrderDetailID = 121317;
-- Clustered Index SCAN — reads the whole leaf level
```

BEST PRACTICE — Choosing a clustered key

A good clustered key is **narrow, unique, static, and ever-increasing** (e.g. an `IDENTITY` or `BIGINT` surrogate). Narrow because the clustered key is copied into every nonclustered index as the row locator. Ever-increasing (like IDENTITY) to avoid page splits from random inserts. Avoid wide keys like `UNIQUEIDENTIFIER` (GUID) as the clustered key — they fragment badly and bloat every nonclustered index.

1.5 Nonclustered index — a sorted copy of selected columns

A **nonclustered index** is a separate B-Tree whose leaf level holds the index key columns plus a **row locator** back to the full row. If the table is clustered, the locator is the **clustered key**; if the table is a heap, the locator is the **RID**. You can create up to 999 nonclustered indexes per table.

Scenario: queries constantly filter orders by `CustomerID`, but the clustered key is `SalesOrderID`. Without help this is a scan.

 © PMSQLDBA

```
SET STATISTICS IO ON;

-- Before: no index on CustomerID -> Clustered Index Scan
SELECT SalesOrderID, OrderDate, TotalDue
FROM   Sales.SalesOrderHeader
WHERE  CustomerID = 29825;
--  logical reads ~700 (scans the whole table)

-- Create a targeted nonclustered index
CREATE NONCLUSTERED INDEX IX_SOH_CustomerID
    ON Sales.SalesOrderHeader (CustomerID);

-- After: Index Seek + Key Lookup, logical reads drop dramatically
SELECT SalesOrderID, OrderDate, TotalDue
FROM   Sales.SalesOrderHeader
WHERE  CustomerID = 29825;
```

The query now **seeks** the nonclustered index on `CustomerID`, but `OrderDate` and `TotalDue` are not in that index — so for each matching row SQL Server jumps back to the clustered index to fetch them. That jump is a **Key Lookup**, and eliminating it is one of the most common tuning moves. We fix it next with covering indexes.

 © PMSQLDBA

1.6 Composite indexes and key-column order

A **composite (multi-column) index** has more than one key column. The order of the columns is the single most important design decision, because the index is sorted by the **left-most column first**, then the next, and so on — exactly like a phone book sorted by (LastName, FirstName).

The practical rule is the **left-most prefix rule**: an index on `(A, B, C)` can efficiently seek on `A`, on `A, B`, and on `A, B, C` — but **not** on `B` alone or `C` alone.

```
CREATE NONCLUSTERED INDEX IX_SOD_Product_Order
    ON Sales.SalesOrderDetail (ProductID, SalesOrderID);

-- SEEK: filters the leading column
SELECT * FROM Sales.SalesOrderDetail WHERE ProductID = 776;                    --
seek
SELECT * FROM Sales.SalesOrderDetail WHERE ProductID = 776 AND SalesOrderID=43659;
-- seek

-- SCAN: skips the leading column, so the sort order is useless
SELECT * FROM Sales.SalesOrderDetail WHERE SalesOrderID = 43659;               --
cannot use this index efficiently
```

Query predicate	Index (ProductID, SalesOrderID)	Why
`ProductID = 776`	Seek	Leading column matched
`ProductID = 776 AND SalesOrderID = ...`	Seek	Left-most prefix fully matched
`SalesOrderID = ...` only	Scan (or ignored)	Leading column skipped
`ProductID > 700 AND SalesOrderID = ...`	Partial seek + residual	Range on lead, second col not seekable

BEST PRACTICE — Ordering columns in a composite index

Put **equality-predicate columns first**, then the **range/inequality column**, then columns used only for output. Also weigh **selectivity**: the column that filters out the most rows generally belongs near the front — but never at the cost of breaking the equality-before-range rule.

1.7 Covering indexes and INCLUDE columns

An index **covers** a query when it contains every column the query needs — in the key or in the **INCLUDE** clause — so SQL Server never has to touch the base table. This eliminates Key/RID Lookups entirely.

Key columns are sorted and can be seeked/ranged. **Included columns** are stored only at the **leaf level**, are not sorted, and do not count toward the 900-byte / 32-column key limit. Use INCLUDE for columns you only need to *return*, not to *filter or sort on*.

```
-- Recall the earlier query still doing a Key Lookup:
--   WHERE CustomerID = 29825  returning OrderDate, TotalDue
```

 © PMSQLDBA

```
-- Covering index: key = filter column, INCLUDE = returned columns
CREATE NONCLUSTERED INDEX IX_SOH_CustomerID_Covering
    ON Sales.SalesOrderHeader (CustomerID)
    INCLUDE (OrderDate, TotalDue);

SET STATISTICS IO ON;
SELECT SalesOrderID, OrderDate, TotalDue
FROM   Sales.SalesOrderHeader
WHERE  CustomerID = 29825;
-- Now: single Index Seek, NO Key Lookup. logical reads minimal.
```

TIP — The trade-off

A covering index makes reads fast but adds write cost and storage: every INCLUDEd column is duplicated in the index and must be maintained on every INSERT/UPDATE/DELETE. Cover the queries that matter most; do not blindly INCLUDE every column.

 © PMSQLDBA

1.8 Unique indexes

A **unique index** enforces that no two rows share the same key value, and it also gives the optimizer valuable information: knowing a column is unique lets it estimate exactly one matching row, which produces better plans. Primary keys and unique constraints are implemented internally as unique indexes.

```
-- Enforce one row per (SalesOrderID, ProductID) business rule
CREATE UNIQUE NONCLUSTERED INDEX UX_SOD_Order_Product
    ON Sales.SalesOrderDetail (SalesOrderID, ProductID);

-- A duplicate insert now fails with a unique-key violation.
-- The optimizer also treats a seek on this key as returning at most 1 row.
```

NOTE — Unique vs. non-unique locators

On a non-unique clustered index, SQL Server silently adds a 4-byte hidden **uniquifier** to duplicate keys. Declaring your clustered key unique when it truly is avoids this overhead and improves estimates.

1.9 Filtered indexes

A **filtered index** indexes only the rows that match a `WHERE` predicate. It is smaller, cheaper to maintain, and highly selective — ideal when queries repeatedly target a small, well-defined subset (active records, a status flag, non-NULL values).

```
-- Many orders are online (OnlineOrderFlag = 1). Suppose reporting
-- almost always queries the smaller set of in-store orders.
CREATE NONCLUSTERED INDEX FIX_SOH_Store_Recent
    ON Sales.SalesOrderHeader (OrderDate)
    INCLUDE (CustomerID, TotalDue)
    WHERE OnlineOrderFlag = 0;

-- The optimizer uses it ONLY when the query's predicate is provably
-- a subset of the filter:
SELECT CustomerID, OrderDate, TotalDue
FROM   Sales.SalesOrderHeader
WHERE  OnlineOrderFlag = 0
  AND  OrderDate >= '2013-01-01';   -- uses FIX_SOH_Store_Recent
```

WARNING — Filtered-index gotchas

The query predicate must match the filter for the index to be used. Also, filtered indexes require specific `SET` options (`ARITHABORT`, `QUOTED_IDENTIFIER`, `ANSI_NULLS` ON, etc.) at both create time and query time, or the index is ignored — and parameterized queries whose value could fall outside the filter may skip it too.

1.10 Columnstore indexes — for analytics

Everything so far has been **rowstore** (B-Tree). A **columnstore index** stores data by **column** instead of by row, compressed in ~1-million-row **rowgroups**. This gives huge compression and lets analytic queries

(SUM/GROUP BY over millions of rows) read only the columns they need in **batch mode**, often 10×–100× faster than rowstore.

Type	Best for	Notes
Clustered Columnstore (CCI)	Large fact / data-warehouse tables	Becomes the entire table storage; great compression
Nonclustered Columnstore (NCCI)	Real-time analytics on an OLTP table	Sits alongside the rowstore clustered index

```
-- Build an analytics copy and add a clustered columnstore index
SELECT * INTO dbo.SalesOrderDetail_CS FROM Sales.SalesOrderDetail;

CREATE CLUSTERED COLUMNSTORE INDEX CCI_SOD_CS
    ON dbo.SalesOrderDetail_CS;

-- Aggregation now uses Batch-mode Columnstore Index Scan
SELECT ProductID, SUM(LineTotal) AS Revenue, COUNT(*) AS Lines
FROM   dbo.SalesOrderDetail_CS
GROUP BY ProductID
ORDER BY Revenue DESC;
-- Columnstore Index Scan + Hash Aggregate, batch execution mode
```

TIP — When NOT to use columnstore

Columnstore shines on scans and aggregations over many rows. It is a poor fit for single-row OLTP lookups and frequent narrow updates. Rule of thumb: rowstore for **OLTP** point queries, columnstore for **OLAP / reporting** aggregates.

 © PMSQLDBA

1.11 Other index types you should recognize

Index type	Purpose	Example use in AdventureWorks
Computed-column index	Index a derived expression	Persist & index `YEAR(OrderDate)` for year filters
Indexed view	Materialize an aggregate/join	Pre-aggregate sales totals by product
XML index	Query into XML columns	`Person.Person.Demographics` XML data
Spatial index	Geometry / geography queries	Store/location distance searches
Full-text index	Linguistic text search	Search `Production.ProductReview.Comments`

Computed-column indexing is the most commonly useful of these. To index an expression, persist it or index it directly (SQL Server can index a deterministic computed column):

```
ALTER TABLE Sales.SalesOrderHeader
    ADD OrderYear AS YEAR(OrderDate) PERSISTED;

CREATE NONCLUSTERED INDEX IX_SOH_OrderYear
    ON Sales.SalesOrderHeader (OrderYear) INCLUDE (TotalDue);

-- Now this is seekable instead of scanning + computing on the fly:
SELECT OrderYear, SUM(TotalDue)
FROM   Sales.SalesOrderHeader
WHERE  OrderYear = 2013
GROUP BY OrderYear;
```

WARNING — Non-SARGable predicates

Wrapping an indexed column in a function — `WHERE YEAR(OrderDate) = 2013` — makes the predicate **non-SARGable**: SQL Server must compute the function for every row and cannot seek. Either index the computed column (as above) or rewrite as a range: `WHERE OrderDate >= '2013-01-01' AND OrderDate < '2014-01-01'`.

1.12 Fill factor, fragmentation, and maintenance

As rows are inserted and updated, index pages **split** and become **fragmented** — logical order no longer matches physical order, and pages have empty space. Fragmentation raises page reads and hurts read-ahead. Two levers control it:

- **FILL FACTOR** — how full each leaf page is packed at build time. A lower fill factor (e.g. 80–90) leaves room for inserts, reducing future splits at the cost of more space.
- **Rebuild vs. Reorganize** — REBUILD recreates the index (removes fragmentation, updates statistics, can be ONLINE in Enterprise). REORGANIZE defragments the leaf in place (always online, lighter).

Avg fragmentation	Recommended action
< 5–10%	Do nothing
10–30%	REORGANIZE (online, incremental)
> 30%	REBUILD (ONLINE if Enterprise)

 © PMSQLDBA

Avg fragmentation	Recommended action

```
-- Reorganize (light, always online)
ALTER INDEX IX_SOH_CustomerID ON Sales.SalesOrderHeader REORGANIZE;

-- Rebuild with a fill factor and online option
ALTER INDEX IX_SOH_CustomerID ON Sales.SalesOrderHeader
    REBUILD WITH (ONLINE = ON, FILLFACTOR = 90, SORT_IN_TEMPDB = ON);

-- Rebuild ALL indexes on a table
ALTER INDEX ALL ON Sales.SalesOrderHeader REBUILD;
```

BEST PRACTICE — Production maintenance

Do not blanket-rebuild everything nightly. Use a threshold-based routine (Ola Hallengren's IndexOptimize is the industry standard) that only reorganizes/rebuilds indexes above the fragmentation and page-count thresholds, and always keep statistics updated. On your VMSQL01/02/03 AG lab, remember REBUILD generates significant log and redo traffic on the secondaries.

1.13 Finding missing, unused, and duplicate indexes

SQL Server tracks index usage and the indexes the optimizer *wished* it had. These DMVs are your starting point for a real workload, but treat "missing index" suggestions as **hints, not commands** — they ignore column order quality, INCLUDE bloat, and write cost.

```
-- 1) Missing index suggestions ranked by impact
SELECT  TOP 25
        ROUND(s.avg_total_user_cost * s.avg_user_impact
              * (s.user_seeks + s.user_scans), 0) AS impact_score,
        d.statement AS [table],
        d.equality_columns, d.inequality_columns, d.included_columns
FROM sys.dm_db_missing_index_group_stats s
JOIN sys.dm_db_missing_index_groups g  ON s.group_handle = g.index_group_handle
JOIN sys.dm_db_missing_index_details d ON g.index_handle = d.index_handle
ORDER BY impact_score DESC;

-- 2) Unused / rarely-used indexes (candidates to drop)
SELECT OBJECT_NAME(i.object_id) AS [table], i.name AS index_name,
       us.user_seeks, us.user_scans, us.user_lookups, us.user_updates
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats us
       ON us.object_id = i.object_id AND us.index_id = i.index_id
      AND us.database_id = DB_ID()
WHERE i.type_desc = 'NONCLUSTERED'
  AND OBJECTPROPERTY(i.object_id,'IsUserTable') = 1
ORDER BY us.user_updates DESC;   -- high writes, zero reads = drop candidate
```

WARNING — Read the fine print on missing-index DMVs

 © PMSQLDBA

These stats reset on service restart, never suggest the ideal **column order**, cannot suggest filtered or columnstore indexes, and can recommend near-duplicate indexes. Always design the final index by hand, then validate with an actual execution plan.

 © PMSQLDBA

Part 2 — Execution Plans

An **execution plan** is the step-by-step recipe the query optimizer chose to run your query. It is the single most important artifact in query tuning: it tells you *what* SQL Server did (seek vs scan, which join, whether it sorted or spilled) and *why* it was slow. If Part 1 taught you the tools, Part 2 teaches you to read the blueprint that shows which tool the engine actually picked.

2.1 The optimizer in one paragraph

SQL Server is a **cost-based optimizer**. For a given query it considers many possible plans, estimates each one's cost using **statistics** (histograms describing data distribution), and picks the cheapest plan it finds within its time budget. "Cost" is a unitless internal estimate, not seconds. Because the estimate depends on statistics, **out-of-date or missing statistics are the number-one cause of bad plans**.

2.2 Estimated vs. Actual plans

	Estimated plan	Actual plan
How to get it	`Ctrl` + `L` (SSMS) — does not run query	`Ctrl` + `M`, then run — includes runtime data
Row counts shown	Estimated only	Estimated AND actual (compare them!)
Runtime warnings	No	Yes — spills, wrong estimates, memory grants
When to use	Quick look at a heavy/long query	Real diagnosis — always prefer this

```
-- Get plans + runtime stats without SSMS GUI:
SET STATISTICS XML ON;      -- returns the actual plan as XML
-- or, richer per-operator timing:
SET STATISTICS PROFILE ON;

SELECT soh.CustomerID, SUM(sod.LineTotal) AS Total
FROM   Sales.SalesOrderHeader soh
JOIN   Sales.SalesOrderDetail sod ON sod.SalesOrderID = soh.SalesOrderID
WHERE  soh.OrderDate >= '2013-01-01'
GROUP BY soh.CustomerID;

SET STATISTICS XML OFF;
```

BEST PRACTICE — Estimated vs Actual = your #1 diagnostic

In the actual plan, hover any operator and compare **Estimated Number of Rows** to **Actual Number of Rows**. A large gap (say 50 estimated, 500,000 actual) means the optimizer was working from bad statistics and almost certainly chose the wrong operators, join type, or memory grant. Fixing the estimate usually fixes the plan.

2.3 How to read a graphical plan

 © PMSQLDBA

Read execution plans **right-to-left** (data flows toward the final SELECT on the left) and **top-to-bottom** for the order in which branches feed a join. Key visual cues:

- **Arrow thickness** — proportional to the number of rows flowing between operators. A fat arrow suddenly appearing means a lot of rows are being produced; a fat arrow feeding a Key Lookup or Sort is a red flag.
- **Operator cost %** — each operator shows its share of the estimated batch cost. The highest-percentage operators are where to focus, but remember these are *estimates*.
- **Warnings (yellow ⚠ / red)** — spills to tempdb, implicit conversions affecting cardinality, missing statistics, excessive memory grant.
- **A green "Missing Index" hint** at the top — a suggestion, not an order (see §1.13).

2.4 Operator properties that matter

Press F4 on any operator to open Properties. The fields you will use constantly:

Property	What it tells you
Estimated vs Actual Rows	Statistics accuracy — the master diagnostic
Estimated / Actual Executions	How many times an operator ran (e.g. inner side of a nested loop)
Logical / Physical Operation	e.g. logical "Inner Join", physical "Nested Loops"
Seek Predicates vs Predicates	Seek Predicate = seekable (uses B-Tree). Predicate = residual filter applied after reading rows
Number of Rows Read	Rows physically read vs. returned — a huge gap signals a residual predicate or scan
Ordered / Warnings / Memory Grant	Whether output is sorted; spill/convert warnings; RAM requested

WARNING — "Predicate" vs "Seek Predicate"

This distinction catches many people. A **Seek Predicate** navigates the index B-Tree — cheap. A plain **Predicate** is a residual filter checked against every row already read — if you see a Seek that read 1,000,000 rows but returned 12, the extra filtering is happening in a residual Predicate, and you likely need a better index.

2.5 Cardinality estimation, statistics, and parameter sniffing

Cardinality estimation (CE) is the optimizer guessing how many rows each step will produce, using column **statistics**. Everything downstream — join type, memory grant, parallelism — flows from those guesses.

```
-- Inspect and refresh statistics
DBCC SHOW_STATISTICS('Sales.SalesOrderDetail', 'PK_SalesOrderDetail...');
UPDATE STATISTICS Sales.SalesOrderDetail WITH FULLSCAN;

-- See when stats were last updated & how stale (modification counter)
```

```
SELECT s.name AS stat_name, sp.last_updated, sp.rows, sp.modification_counter
FROM   sys.stats s
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
WHERE  s.object_id = OBJECT_ID('Sales.SalesOrderDetail');
```

Parameter sniffing: on first execution of a stored procedure, SQL Server "sniffs" the passed parameter values, builds a plan optimized for *those* values, and caches it. If later calls pass values with very different selectivity, the cached plan can be badly wrong — fast for the first customer, slow for the next.

```
-- Remedies for a bad sniffed plan:

-- (a) Recompile each run (fresh plan every time; small CPU cost)
SELECT ... FROM ... WHERE CustomerID = @cust
OPTION (RECOMPILE);

-- (b) Optimize for a representative/unknown value
OPTION (OPTIMIZE FOR (@cust = 29825));
OPTION (OPTIMIZE FOR UNKNOWN);

-- (c) Force a known-good plan
OPTION (USE PLAN N'...');   -- or Query Store "Force Plan" (preferred)
```

TIP — Query Store is the modern tool

Since SQL Server 2016, **Query Store** records every plan and its runtime for each query, lets you spot regressions, and lets you **force** a known-good plan with one click — no code change. Enable it (`ALTER DATABASE ... SET QUERY_STORE = ON`) on any database you tune. It is the cleanest fix for parameter-sniffing regressions in production.

2.6 Plan cache and reuse

Compiled plans are cached and reused to avoid re-optimizing every call. You can inspect the cache to find your most expensive queries:

```
-- Top queries by total logical reads (I/O offenders)
SELECT TOP 20
       qs.total_logical_reads,
       qs.execution_count,
       qs.total_logical_reads / qs.execution_count AS avg_reads,
       SUBSTRING(st.text, (qs.statement_start_offset/2)+1,
           ((CASE qs.statement_end_offset WHEN -1 THEN DATALENGTH(st.text)
             ELSE qs.statement_end_offset END - qs.statement_start_offset)/2)+1)
AS query_text,
       qp.query_plan
FROM   sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
ORDER BY qs.total_logical_reads DESC;
```

 © PMSQLDBA

Part 3 — Plan Operators, Explained

Operators are the individual steps inside a plan. Learning what each one does — and, crucially, what its presence usually *means* about your query and indexes — is what turns plan-reading from guesswork into diagnosis. This part groups the operators you will actually meet and pairs each with an AdventureWorks query that produces it.

3.1 Data-access operators: scans vs. seeks

Operator	What it does	Usually means
Table Scan	Reads every page of a **heap**	No clustered index; no usable index for the predicate
Clustered Index Scan	Reads the whole clustered leaf (the table)	No usable nonclustered index, or query needs most rows
Index Scan (NC)	Reads the whole nonclustered leaf	Index chosen but predicate not seekable, or covering scan
Index Seek	Navigates the B-Tree to the rows	Good — a SARGable predicate on a leading key column
Columnstore Index Scan	Batch-mode scan of a columnstore	Analytic aggregation — this is the goal for OLAP

Scan is not always bad, seek is not always good. A scan that reads a small table, or a query that genuinely needs 80% of the rows, is fine. A "seek" that then does a million-row residual filter or a Key Lookup per row can be worse than a clean scan. Always look at rows read vs. rows returned.

```
-- SEEK: SARGable predicate on the clustered leading key
SELECT * FROM Sales.SalesOrderDetail WHERE SalesOrderID = 43659;    -- Clustered
Index Seek

-- SCAN: function on the column makes it non-SARGable
SELECT * FROM Sales.SalesOrderHeader WHERE YEAR(OrderDate) = 2013;  -- Clustered
Index Scan

-- SEEK: same intent, rewritten SARGable as a range
SELECT * FROM Sales.SalesOrderHeader
WHERE  OrderDate >= '2013-01-01' AND OrderDate < '2014-01-01';      -- Seek (with
a supporting index)
```

3.2 Lookups: Key Lookup and RID Lookup

A **lookup** happens when a nonclustered index found the rows but does not contain every column the query needs, so SQL Server jumps back to the full row for the rest. On a **clustered** table this is a **Key Lookup**; on a **heap** it is a **RID Lookup**. Each lookup is one B-Tree navigation *per row*, so a lookup on the fat arrow (many rows) is expensive.

 © PMSQLDBA

```
-- Force a Key Lookup: seek on CustomerID, but return columns not in the index
CREATE NONCLUSTERED INDEX IX_SOH_Cust ON Sales.SalesOrderHeader (CustomerID);

SELECT CustomerID, OrderDate, TotalDue, Status   -- OrderDate/TotalDue/Status not in index
FROM   Sales.SalesOrderHeader
WHERE  CustomerID = 29825;
-- Plan: Index Seek  +  Key Lookup (Nested Loops)

-- Eliminate the lookup by COVERING the query:
CREATE NONCLUSTERED INDEX IX_SOH_Cust_Cover
    ON Sales.SalesOrderHeader (CustomerID) INCLUDE (OrderDate, TotalDue, Status);
-- Plan: single Index Seek, lookup gone
```

BEST PRACTICE — The lookup-elimination pattern

See a Key Lookup fed by a thick arrow? INCLUDE the returned columns in the nonclustered index so it covers the query. For a rarely-run query returning few rows, a lookup is fine and not worth the extra index maintenance — cover the hot paths, not everything.

 © PMSQLDBA

3.3 Physical join operators

SQL Server has three ways to physically execute a logical join. The optimizer picks one based on the estimated row counts, sort order, and available indexes. Knowing why each was chosen tells you whether the estimate (and therefore the choice) is trustworthy.

Join operator	How it works	Optimizer picks it when…
Nested Loops	For each row of the outer (small) input, seek the inner input	One input is small AND the inner side is indexed on the join key
Merge Join	Both inputs sorted on the join key, walked in lockstep	Both inputs already sorted on the key (e.g. by indexes)
Hash Match	Build a hash table from one input, probe with the other	Large, unsorted inputs with no useful index

Nested Loops

```
-- Small outer set (one customer's orders) + indexed inner seek -> Nested Loops
SELECT soh.SalesOrderID, sod.ProductID, sod.LineTotal
FROM   Sales.SalesOrderHeader soh
JOIN   Sales.SalesOrderDetail sod ON sod.SalesOrderID = soh.SalesOrderID
WHERE  soh.CustomerID = 29825;   -- few outer rows -> Nested Loops with inner
Seek
```

Efficient when the outer input is small and the inner side can be seeked. Dangerous when the estimate is too low: if the optimizer thinks the outer set is 10 rows but it is really 2,000,000, it will loop-and-seek millions of times — a classic symptom of stale statistics.

Merge Join

```
-- Both sides ordered by the join key (clustered on SalesOrderID) -> Merge Join
SELECT soh.SalesOrderID, sod.SalesOrderDetailID
FROM   Sales.SalesOrderHeader soh
JOIN   Sales.SalesOrderDetail sod ON sod.SalesOrderID = soh.SalesOrderID
ORDER BY soh.SalesOrderID;   -- pre-sorted inputs favor a Merge Join
```

Very efficient when both inputs are already sorted on the join key. If they are not, the optimizer may inject a **Sort** (expensive) to enable the merge — watch for that.

Hash Match

```
-- Large unsorted inputs, no covering index on the join -> Hash Match
SELECT p.ProductID, SUM(sod.LineTotal) AS Revenue
FROM   Production.Product p
JOIN   Sales.SalesOrderDetail sod ON sod.ProductID = p.ProductID
GROUP BY p.ProductID;   -- Hash Match (join) + Hash Match (aggregate)
```

WARNING — Hash Match warning signs

A Hash Match building from a **huge** input needs a large memory grant. If the grant is under-estimated (bad stats again), the hash **spills to tempdb** — you will see a spill warning on the operator and a

 © PMSQLDBA

sharp slowdown. Hash joins on small tables can also signal a missing index that would have enabled a cheaper Nested Loops or Merge.

 © PMSQLDBA

3.4 Sort, aggregate, and spool operators

Operator	What it does	Watch for
Sort	Orders rows (ORDER BY, merge join, stream agg)	Expensive + blocking; can **spill to tempdb**. An index in the right order removes it
Stream Aggregate	Aggregates a **sorted** input	Cheap, but requires sorted input (often paired with a Sort)
Hash Aggregate	Aggregates via a hash table (unsorted input)	Big GROUP BY; can spill like a hash join
Table/Index Spool	Caches an intermediate result in tempdb	Eager Spool can hide a self-join or Halloween protection; heavy tempdb use
Compute Scalar	Evaluates an expression per row	Usually cheap; a non-SARGable expression here can force a scan upstream

```
-- Sort caused by ORDER BY on a non-indexed column
SELECT SalesOrderID, TotalDue
FROM   Sales.SalesOrderHeader
ORDER BY TotalDue DESC;        -- Sort operator appears

-- Remove the Sort by providing a pre-sorted index
CREATE NONCLUSTERED INDEX IX_SOH_TotalDue ON Sales.SalesOrderHeader (TotalDue
DESC);
-- Now the plan reads the index in order -> no Sort operator
```

TIP — Sort is a blocking, memory-hungry operator

A Sort must consume its **entire** input before producing the first output row, and it requests a memory grant sized from the row estimate. Under-estimate the rows and it **spills to tempdb**, turning a fast query slow. The fix is almost always an index that already delivers rows in the required order.

3.5 Parallelism and remaining operators

When a plan's estimated cost exceeds the **cost threshold for parallelism** and enough schedulers are free, SQL Server splits work across threads. You will see **Parallelism (Gather Streams / Repartition Streams / Distribute Streams)** operators and a small yellow circle-arrows badge on parallel operators.

Operator	Meaning
Parallelism (Gather Streams)	Combines parallel threads back into one stream (top of a parallel plan)
Filter	Applies a WHERE predicate that could not be pushed into a seek
Top	Returns the first N rows (TOP / EXISTS short-circuit)
Concatenation	Combines inputs for UNION ALL
Assert	Enforces a constraint (e.g. check / FK / uniqueness)
Nested Loops (Left Semi Join)	Physical shape used for EXISTS / IN sub-queries

© PMSQLDBA

Operator	Meaning

NOTE — Parallelism is a symptom, not a cause

A big parallel plan often means a big scan or a bad estimate inflated the cost past the threshold. Before changing `MAXDOP` or the cost threshold, check whether an index would shrink the work so parallelism is no longer needed. `CXPACKET`/`CXCONSUMER` waits indicate parallel queries waiting on each other — sometimes a tuning signal, sometimes normal for analytic workloads.

© PMSQLDBA

Part 4 — Real-Time Tuning Scenarios

These are end-to-end walkthroughs of the situations you will actually face. Each follows the same rhythm: **symptom → diagnose with the plan → fix → verify**. Run each in AdventureWorks2019 with `SET STATISTICS IO ON` and the actual plan (`Ctrl`+`M`) so you can watch the numbers move.

Scenario 1 — A dashboard query doing a Key Lookup on a fat arrow

Symptom: a customer-orders widget times out under load. The plan shows an Index Seek feeding a Key Lookup, with a thick arrow between them.

```
SET STATISTICS IO ON;
CREATE NONCLUSTERED INDEX IX_SOH_Cust ON Sales.SalesOrderHeader (CustomerID);

SELECT CustomerID, OrderDate, ShipDate, TotalDue, Status
FROM   Sales.SalesOrderHeader
WHERE  CustomerID = 29825;
-- BEFORE: Index Seek + Key Lookup.  logical reads high, one lookup per row.
```

Diagnose: the seek found rows by `CustomerID`, but `OrderDate/ShipDate/TotalDue/Status` are not in that index, so each row triggers a Key Lookup into the clustered index.

Fix: cover the query — put the filter column in the key and the returned columns in INCLUDE.

```
CREATE NONCLUSTERED INDEX IX_SOH_Cust_Cover
    ON Sales.SalesOrderHeader (CustomerID)
    INCLUDE (OrderDate, ShipDate, TotalDue, Status);

SELECT CustomerID, OrderDate, ShipDate, TotalDue, Status
FROM   Sales.SalesOrderHeader
WHERE  CustomerID = 29825;
-- AFTER: single Index Seek, NO Key Lookup. logical reads collapse.
```

BEST PRACTICE — Verify, do not assume

Compare `logical reads` in the Messages tab before and after. The lookup elimination should cut them by an order of magnitude. If the arrow was thin (few rows), the lookup was harmless and the extra index may not be worth the write cost.

Scenario 2 — Implicit conversion silently killing a seek

Symptom: a lookup by account/reference number scans instead of seeks. The plan shows an **implicit conversion** warning (⚠) on the operator and a `CONVERT_IMPLICIT` in the Predicate.

```
-- Suppose AccountNumber is VARCHAR but the app passes an NVARCHAR parameter.
DECLARE @acct NVARCHAR(15) = N'10-4020-000609';

SELECT CustomerID, AccountNumber
```

 © PMSQLDBA

```
FROM   Sales.SalesOrderHeader
WHERE  AccountNumber = @acct;     -- N'...' forces CONVERT_IMPLICIT on the column
-- Plan: Index Scan + yellow implicit-conversion warning
```

Diagnose: because NVARCHAR has higher data-type precedence than VARCHAR, SQL Server converts the **column** (every row) to NVARCHAR, which is non-SARGable — the seek is lost and cardinality estimates degrade.

Fix: match the parameter type to the column type (or the column to the parameter). The correct type restores the seek.

```
DECLARE @acct VARCHAR(15) = '10-4020-000609';   -- match the column type
SELECT CustomerID, AccountNumber
FROM   Sales.SalesOrderHeader
WHERE  AccountNumber = @acct;     -- Index Seek, warning gone
```

WARNING — This one hides in ORMs

Entity Framework and other ORMs default string parameters to NVARCHAR. Against VARCHAR columns this produces exactly this implicit-conversion scan across an entire application. Align column and parameter types, or map the ORM column to VARCHAR.

 © PMSQLDBA

Scenario 3 — Non-SARGable date filter forcing a scan

Symptom: a "sales in 2013" report scans the whole table even though an index on `OrderDate` exists.

```
SET STATISTICS IO ON;
SELECT COUNT(*), SUM(TotalDue)
FROM   Sales.SalesOrderHeader
WHERE  YEAR(OrderDate) = 2013;     -- function on the column = non-SARGable = SCAN
```

Diagnose: wrapping `OrderDate` in `YEAR()` means SQL Server must compute the function for every row; it cannot use the sorted index to seek a range.

Fix: rewrite the predicate as a half-open range so the index range-seeks.

```
CREATE NONCLUSTERED INDEX IX_SOH_OrderDate ON Sales.SalesOrderHeader (OrderDate)
    INCLUDE (TotalDue);

SELECT COUNT(*), SUM(TotalDue)
FROM   Sales.SalesOrderHeader
WHERE  OrderDate >= '2013-01-01'
  AND  OrderDate <  '2014-01-01';   -- SARGable range -> Index Seek
```

TIP — The half-open range pattern

Use `>= start AND < next-period-start` rather than `BETWEEN ... AND '2013-12-31'`. The half-open form is correct to the millisecond, avoids boundary bugs with time components, and stays perfectly SARGable.

Scenario 4 — Parameter sniffing: fast for one customer, slow for another

Symptom: a stored procedure runs in 20 ms for most customers but 8 seconds for a few. The cached plan was built for a low-volume customer (favoring Nested Loops + seeks) and is reused for a high-volume one where a Hash Match would be better.

```
CREATE OR ALTER PROCEDURE dbo.GetCustomerOrders @CustomerID INT
AS
SELECT soh.SalesOrderID, sod.ProductID, sod.LineTotal
FROM   Sales.SalesOrderHeader soh
JOIN   Sales.SalesOrderDetail sod ON sod.SalesOrderID = soh.SalesOrderID
WHERE  soh.CustomerID = @CustomerID;
GO
EXEC dbo.GetCustomerOrders 29825;   -- sniffs & caches a plan for THIS value
```

Diagnose: compare estimated vs actual rows across executions — the estimate matches the sniffed customer, not the current one. In Query Store you will see one query with two very different runtime profiles on the same plan.

 © PMSQLDBA

Fix options (pick per situation):

1. **Recompile per call** — freshest plan, small CPU cost, good when data varies wildly:

```
WHERE soh.CustomerID = @CustomerID OPTION (RECOMPILE);
```

2. **OPTIMIZE FOR** a representative or unknown value — one stable plan:

```
OPTION (OPTIMIZE FOR (@CustomerID = 29825));   -- or OPTIMIZE FOR UNKNOWN
```

3. **Force a known-good plan in Query Store** — no code change, best for production:

```
-- In SSMS: Query Store > Top Resource Consuming Queries > select the good plan >
Force Plan
-- Or programmatically:
EXEC sp_query_store_force_plan @query_id = 42, @plan_id = 17;
```

BEST PRACTICE — Diagnose before you medicate

Do not sprinkle `OPTION (RECOMPILE)` everywhere. First confirm sniffing is the cause (estimate matches a *different* parameter than the one running). For a high-frequency proc, RECOMPILE's per-call compilation can cost more than the bad plan. Query Store plan forcing is usually the safest production fix.

 © PMSQLDBA

Scenario 5 — A Sort spilling to tempdb

Symptom: a paged/sorted grid is slow and the plan shows a **Sort** operator with a yellow spill warning; tempdb usage spikes under load.

```
SET STATISTICS IO ON;
SELECT SalesOrderID, CustomerID, TotalDue
FROM   Sales.SalesOrderHeader
ORDER BY TotalDue DESC;      -- Sort operator; may spill if the grant is under-
estimated
```

Diagnose: the Sort requested a memory grant sized from the row estimate. If the estimate was low (stale stats) the sort **spills** — it writes intermediate data to tempdb and reads it back, adding physical I/O.

Fix: provide an index already sorted in the required order so the Sort disappears entirely; and update statistics so any remaining grants are sized correctly.

```
CREATE NONCLUSTERED INDEX IX_SOH_TotalDue_Desc
    ON Sales.SalesOrderHeader (TotalDue DESC) INCLUDE (CustomerID);

UPDATE STATISTICS Sales.SalesOrderHeader WITH FULLSCAN;

SELECT SalesOrderID, CustomerID, TotalDue
FROM   Sales.SalesOrderHeader
ORDER BY TotalDue DESC;      -- reads the index in order: NO Sort, no spill
```

Scenario 6 — OLAP aggregation: rowstore scan vs. columnstore

Symptom: a management report summing revenue by product across the whole order-detail table takes seconds on a rowstore clustered index scan.

```
-- Rowstore: full Clustered Index Scan + Hash Aggregate (row mode)
SELECT ProductID, SUM(LineTotal) AS Revenue, COUNT(*) AS Lines
FROM   Sales.SalesOrderDetail
GROUP BY ProductID
ORDER BY Revenue DESC;
```

Fix: for analytic aggregation over many rows, a columnstore index reads only the needed columns in **batch mode** with heavy compression.

```
SELECT * INTO dbo.SalesOrderDetail_CS FROM Sales.SalesOrderDetail;
CREATE CLUSTERED COLUMNSTORE INDEX CCI_SOD ON dbo.SalesOrderDetail_CS;

SELECT ProductID, SUM(LineTotal) AS Revenue, COUNT(*) AS Lines
FROM   dbo.SalesOrderDetail_CS
GROUP BY ProductID
ORDER BY Revenue DESC;
-- Columnstore Index Scan (batch mode) + Hash Aggregate -> far less CPU & I/O
```

NOTE — Right tool, right workload

 © PMSQLDBA

> Keep the rowstore clustered index for OLTP point lookups and add a columnstore only for the analytic path (a nonclustered columnstore lets both coexist on one table for real-time operational analytics). Do not replace OLTP indexes with columnstore.

 © PMSQLDBA

Scenario 7 — A repeatable tuning workflow

Tie it all together with a checklist you can run on any slow query. This is the same loop whether you are on a client's production box or your VMSQL01/02/03 lab.

1. **Capture the actual plan** (`Ctrl`+`M`) with `SET STATISTICS IO, TIME ON`. Record baseline reads and elapsed time.
2. **Find the expensive operator** — highest cost %, thickest arrows, any yellow/red warnings.
3. **Check estimate vs actual rows** (`F4`). A big gap ⇒ fix statistics first (`UPDATE STATISTICS ... WITH FULLSCAN`).
4. **Classify the problem**: Scan where a Seek belongs? Key/RID Lookup on a fat arrow? Sort/Hash spill? Implicit conversion? Non-SARGable predicate?
5. **Apply the matching fix**: covering/filtered index, SARGable rewrite, type alignment, right join-enabling index, or a sniffing remedy.
6. **Re-run and compare** the same metrics. Keep the change only if reads/time actually improved.
7. **Weigh the write cost** of any new index against its read benefit before shipping it, and confirm it does not duplicate an existing index.

> **BEST PRACTICE — Golden rules to carry into any interview or war room**
> 1) Reduce **page reads**, not just "make it look like a seek".
> 2) A plan is only as good as its **statistics**.
> 3) **Cover** hot queries, but do not over-index — every index is a write tax.
> 4) Keep predicates **SARGable**.
> 5) Prefer **Query Store** for regressions.
> 6) Always **measure before and after** — the plan tells you the story, `STATISTICS IO/TIME` proves the win.

 © PMSQLDBA

Appendix — Quick Reference

A. Index decision cheat-sheet

If you need to…	Use
Define physical order / one per table	Clustered index (narrow, unique, ever-increasing key)
Speed up a filter on non-key columns	Nonclustered index on the filter column(s)
Eliminate a Key/RID Lookup	Covering index (filter cols in key, returned cols in INCLUDE)
Filter/sort on multiple columns	Composite index (equality cols first, then range)
Enforce uniqueness + better estimates	Unique index
Index only a subset of rows	Filtered index (WHERE clause on the index)
Aggregate over millions of rows	Columnstore index (clustered for DW, NC for hybrid)
Index a derived expression	PERSISTED computed column + index

B. Operator → likely fix

See this operator	First thing to check / fix
Table Scan	Add a clustered index; add a usable nonclustered index
Clustered/Index Scan	Is the predicate SARGable? Is a seekable index missing?
Key / RID Lookup (fat arrow)	Cover the query with INCLUDE columns
Hash Match (huge inputs)	Check estimates; index to enable Nested Loops/Merge; watch spills
Sort (with spill)	Provide a pre-sorted index; update statistics
Nested Loops (millions of executions)	Under-estimate ⇒ fix statistics; the outer set is bigger than thought
Implicit conversion warning	Align parameter and column data types
Parallelism on a simple query	A scan/bad estimate inflated cost — index it down

C. Essential tuning commands

```
-- Measurement
SET STATISTICS IO ON;  SET STATISTICS TIME ON;
SET STATISTICS XML ON;                 -- actual plan as XML

-- Statistics
UPDATE STATISTICS schema.table WITH FULLSCAN;
DBCC SHOW_STATISTICS('schema.table','stat_or_index_name');
SELECT * FROM sys.dm_db_stats_properties(OBJECT_ID('schema.table'), 1);

-- Index maintenance
ALTER INDEX ALL ON schema.table REORGANIZE;
ALTER INDEX ALL ON schema.table REBUILD WITH (ONLINE = ON, FILLFACTOR = 90);
```

 © PMSQLDBA

```
-- Physical / fragmentation
SELECT * FROM
sys.dm_db_index_physical_stats(DB_ID(),OBJECT_ID('schema.table'),NULL,NULL,'LIMITED'
);

-- Usage / missing indexes
SELECT * FROM sys.dm_db_index_usage_stats     WHERE database_id = DB_ID();
SELECT * FROM sys.dm_db_missing_index_details;

-- Query Store (regressions & plan forcing)
ALTER DATABASE AdventureWorks2019 SET QUERY_STORE = ON;
EXEC sp_query_store_force_plan @query_id = <id>, @plan_id = <id>;
```

End of document. *All examples target the AdventureWorks2019 sample database on SQL Server 2019+. Validate every index change against an actual execution plan and STATISTICS IO/TIME before promoting it beyond your VMSQL lab.*

 © PMSQLDBA